



# Phase-wise Implementation

Resume ingestion, RAG retrieval and AI candidate search

|             |                                                                                                                                               |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Repository  | <a href="https://github.com/rathna070-ai/HR_-AI-Assistance_with-Rag">https://github.com/rathna070-ai/HR_-AI-Assistance_with-Rag</a> (private) |
| Date        | 7 October 2026                                                                                                                                |
| Backend     | Node.js · TypeScript · Express 5 · MongoDB Atlas                                                                                              |
| AI services | Mistral mistral-embed · Groq openai/gpt-oss-120b                                                                                              |
| Frontend    | React 19 · Vite 8 · TypeScript · Tailwind CSS                                                                                                 |

# Contents

---

1. Architecture
  2. Implementation timeline
  3. Part A — Ingestion pipeline (Phases 1–16 and batch ingestion)
  4. Part B — Retrieval pipeline (Phases 1–19)
  5. Part C — Web frontend (Phases 1–15)
  6. Part D — Rebrand and UX (TalentLens AI)
  7. Part E — AI Search enhancements
  8. API reference
  9. Verification summary
- 

## 1. Architecture

---

```
TalentLens AI web app (recruitbot-web, Vite, port 5173)
/           candidate search chat (AI Search, Vector, BM25, Hybrid)
/ingestion  resume upload
/help       how to use / how it works
|           | /v1/* and /health through the Vite dev proxy
▼
Express server (src/server.ts, port 3000)
middleware: requestId → logger → routes → errorHandler
Ingestion  src/routes/ingestionRoutes.ts           /v1/resume/*
Resumes    src/routes/resumeRoutes.ts              /v1/resumes, /v1/resume/search
Retrieval  src/modules/retrieval/                  /v1/search*, /v1/embeddings
|
├─ MongoDB Atlas   resumes, ingestion_files, resume_vector, resume_bm25
├─ Mistral API     mistral-embed (1024)
└─ Groq API        gpt-oss-120b (primary key → fallback key on 429)
```

### End-to-end flow

```
INGESTION  PDF upload / resume folder (batch)
           → duplicate check (SHA-256 of the file)
           → text extraction (pdf-parse, OCR fallback; Word in batch)
           → cleaning → LLM parsing (or algorithm parser)
           → Mistral embedding → MongoDB (resumes)

RETRIEVAL  query → BM25 || vector (parallel)
           → merge (by resume) → de-duplicate by person
           → LLM re-rank (score + reason) → top K
           → shortlist + per-candidate summaries

UI         AI Search results with pipeline counts, reasons, summaries,
           candidate profile; Vector / BM25 / Hybrid for comparison
```

## 2. Implementation timeline

| Date        | Stage                                  | Outcome                                                                                                                                                                      |
|-------------|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 27 Sep 2026 | Ingestion Phases 1–4                   | Upload API, PDF text extraction, text cleaning, each verified in Postman before the next phase                                                                               |
| 29 Sep 2026 | Ingestion Phases 5–13, batch ingestion | Parsers, skills, LLM parser, embeddings, MongoDB storage, orchestration, errors, logging; batch ingestion of the resume folder in batches of 5 or 10                         |
| 1 Oct 2026  | Ingestion Phases 14–16                 | Vector index, semantic search, resume management APIs                                                                                                                        |
| 3 Oct 2026  | Retrieval Phases 1–18                  | BM25, vector, hybrid, LLM re-ranking, summaries, end-to-end pipeline, fallbacks, validation, tests; Postman collections; first push to GitHub (commit <code>8b61901</code> ) |
| 4 Oct 2026  | Frontend Phases 1–15                   | React app for ingestion and retrieval, one commit per phase ( <code>d509ae1</code> ... <code>3e73892</code> )                                                                |
| 4 Oct 2026  | Rebrand and UX                         | TalentLens AI name, logo and palette; in-app guide; Clear next to Send; larger mode badge ( <code>e1ee54e</code> )                                                           |
| 7 Oct 2026  | AI Search enhancements                 | Re-ranking details, person-level de-duplication, shortlist summaries, end-to-end UI, messages, responsive design ( <code>fc0e8a2</code> )                                    |

## 3. Part A — Ingestion pipeline

Guide: `Resume_Ingestion_Architecture_Phasewise.md`. Verification: `postman/HR-Ingestion.postman_collection.json` (one folder per phase).

| Phase                  | Goal                                        | Implementation                                                                                                            | Endpoint / files                                                                                                   |
|------------------------|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| 1. Project setup       | Ingestion module inside the existing server | Routes, controller, services, repository, config and utils folders; <code>npm run dev</code> runs one server on port 3000 | <code>src/routes</code> , <code>src/controllers</code> , <code>src/services</code> , <code>src/repositories</code> |
| 2. PDF upload API      | Accept resumes securely                     | Multer, PDF only (MIME + extension + content check), max 5 MB, stored in <code>uploads/</code>                            | <code>POST /v1/resume/upload</code> , <code>src/config/multerConfig.ts</code>                                      |
| 3. PDF text extraction | Raw text from the PDF                       | <code>pdf-parse</code> ; OCR with <code>tesseract.js</code> when a PDF has no text layer (scanned)                        | <code>POST /v1/resume/extract</code> , <code>ResumeParserService.ts</code>                                         |
| 4. Text cleaning       | Normalised text                             | Extra spaces, duplicate lines, line breaks and broken ligature characters fixed                                           | <code>POST /v1/resume/clean</code> , <code>src/utils/textCleaner.ts</code>                                         |
| 5. Algorithm parser    | Structured JSON without an LLM              | Name, email, phone, location, skills, company, role, education, total experience                                          | <code>POST /v1/resume/parse</code> , <code>AlgorithmResumeParser.ts</code>                                         |
| 6. Regex utilities     | Shared patterns                             | Email, phone and experience regexes                                                                                       | <code>src/utils/regex.ts</code>                                                                                    |

| Phase                   | Goal                                        | Implementation                                                                                                                                                                          | Endpoint / files                                                                                                         |
|-------------------------|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| 7. Skills detection     | Skills from a dictionary                    | Whole-word matching against <code>src/config/skills.ts</code> , canonical spelling                                                                                                      | <code>POST /v1/resume/skills</code>                                                                                      |
| 8. LLM parser           | Better parsing via <code>.env</code> switch | Groq LLM parser ( <code>USE_LLM_PARSER=true</code> , now the default): adds job titles and an experience summary, rejects non-resumes ("Not a resume"), prompt versioned for re-parsing | <code>POST /v1/resume/llm-parse</code> , <code>LLMResumeParser.ts</code>                                                 |
| 9. Mistral embedding    | Vector for each resume                      | <code>mistral-embed</code> , 1024 dimensions; input = name, role, job titles, skills, company, summary and text (capped)                                                                | <code>POST /v1/resume/embed</code> , <code>EmbeddingService.ts</code>                                                    |
| 10. MongoDB ingestion   | Store resume + embedding                    | <code>resumes</code> document with parsed fields, <code>fileHash</code> , <code>parser</code> , text source, embedding model and dimension                                              | <code>ResumeIngestionRepository.ts</code>                                                                                |
| 11. Ingestion service   | One call does everything                    | Duplicate check (SHA-256) → extract → clean → parse → embed → store; returns <code>ingested</code> (201) or <code>duplicate</code> (200) with timings                                   | <code>POST /v1/resume/inject</code> , <code>ResumeIngestionService.ts</code>                                             |
| 12. Error handling      | Clear failures                              | Only PDF allowed / File too large (400), Resume extraction failed / Not a resume (422), LLM or embedding failure (502), ingestion failed (500)                                          | <code>ingestionController.ts</code> , <code>ingestionRoutes.ts</code>                                                    |
| 13. Logging             | Timings per step                            | JSON log with <code>requestId</code> , file name and extract / parse / embedding / MongoDB times                                                                                        | <code>ResumeIngestionService.ts</code>                                                                                   |
| 14. Vector search index | Searchable embeddings                       | Atlas Vector Search index <code>resume_vector</code> (cosine, 1024) with <code>skills</code> and <code>totalExperience</code> filters; script waits until queryable                     | <code>npm run search:index</code>                                                                                        |
| 15. Semantic search     | First search endpoint                       | Query embedding → <code>\$vectorSearch</code> (local cosine fallback while the index builds); filters for skills, experience, location                                                  | <code>POST /v1/resume/search</code>                                                                                      |
| 16. Resume management   | List, view, delete                          | Paginated list with filters, full document by id (no embedding), delete with <code>ingestion_files</code> clean-up                                                                      | <code>GET /v1/resumes</code> , <code>GET/DELETE /v1/resumes/:id</code>                                                   |
| Batch ingestion         | Ingest the resume folder                    | Batches of 5 or 10, 2 files in parallel, per-file status in <code>ingestion_files</code> so runs resume where they stopped; PDF, Word and HTML-as-.doc supported                        | <code>npm run ingest:batch</code> , <code>POST /v1/resume/batch-inject</code> , <code>GET /v1/resume/batch-status</code> |

## 4. Part B — Retrieval pipeline

Module: `src/modules/retrieval/`. Verification: `postman/HR-Retrieval.postman_collection.json` (Phase 1–19 folders) and `tests/unit`, `tests/integration`.

| Phase                    | Goal                             | Implementation                                                                                                                                                        | Endpoint / files                                                           |
|--------------------------|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| 1. Readiness check       | Search only when data is ready   | 200 when stored embeddings match the configured model and dimension, otherwise 503 with a reason                                                                      | <code>GET /v1/search/readiness</code>                                      |
| 2. Module scaffold       | Retrieval inside the same server | Routes, controller, services, repository, types, utils under <code>src/modules/retrieval</code>                                                                       | <code>retrievalRoutes.ts</code>                                            |
| 3. Query embedding       | Same model as ingestion          | Embeds a query on demand; other models rejected                                                                                                                       | <code>POST /v1/embeddings</code>                                           |
| 4. Resume repository     | Read-only data access            | <code>\$search</code> and <code>\$vectorSearch</code> queries returning the same projection (name, contact, role, company, skills, experience, 600-character snippet) | <code>ResumeRepository.ts</code>                                           |
| 5. Atlas Search / BM25   | Keyword search                   | Index <code>resume_bm25</code> ( <code>lucene.standard</code> ) over text, skills, job titles, experience summary, role and company; matched skills reported          | <code>POST /v1/search/bm25</code> , <code>npm run search:bm25-index</code> |
| 6. Vector search         | Semantic search                  | Query embedding → <code>\$vectorSearch</code> ( <code>numCandidates = max(10 × topK, 100)</code> ), optional exact cosine re-score, minimum-experience filter         | <code>POST /v1/search/vector</code>                                        |
| 7. SearchService methods | Shared search code               | <code>bm25Search</code> and <code>vectorSearch</code> returning one candidate shape with sources                                                                      | <code>SearchService.ts</code> , <code>candidateMapper.ts</code>            |
| 8. Hybrid search         | Both in parallel                 | BM25 and vector run independently; both lists returned side by side; degraded flag if one fails                                                                       | <code>POST /v1/search/hybrid</code>                                        |
| 9. Merge + deduplicate   | One candidate pool               | Lists interleaved, merged by <code>resumeld</code> keeping both scores and sources, snippets capped                                                                   | <code>utils/deduplicate.ts</code> ( <code>mergeCandidates</code> )         |
| 10. Groq LLM service     | LLM access                       | Re-ranking, candidate summaries and metadata extraction with JSON validation, retries and fallback key                                                                | <code>LLMService.ts</code> , <code>GroqClient.ts</code>                    |
| 11. Re-ranking endpoint  | LLM ordering                     | Relevance 0–1 and a one-sentence reason per candidate; invented or repeated ids dropped                                                                               | <code>POST /v1/search/rerank</code>                                        |
| 12. Summarization        | Candidate fit text               | Short or detailed summary grounded in the candidate data, no gendered pronouns                                                                                        | <code>POST /v1/search/summarize</code>                                     |
| 13. End-to-end service   | Full pipeline                    | Validate → BM25 + vector → merge → top N → re-rank → top K → optional summaries                                                                                       | <code>SearchService.endToEndSearch</code>                                  |

| Phase                     | Goal                         | Implementation                                                                                                                                                                                             | Endpoint / files                                                      |
|---------------------------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| 14. Final search endpoint | Public API                   | Options for each stage ( <code>bm25TopK</code> , <code>vectorTopK</code> , <code>rerankTopN</code> , <code>finalTopK</code> , <code>summarize</code> , <code>style</code> , <code>tokens</code> )          | <code>POST /v1/search</code>                                          |
| 15. Fallback logic        | Never fail when a part works | BM25-only or vector-only results, BM25 order when re-ranking fails, results kept when summaries fail; <code>degraded</code> and <code>warnings</code>                                                      | <code>SearchService.ts</code>                                         |
| 16. Logging + timings     | Observability                | Request id (echoed in <code>X-Request-Id</code> ) and per-component timings in every log line                                                                                                              | <code>src/middleware/requestId.ts</code> , <code>logger.ts</code>     |
| 17. Validation controls   | Safe inputs                  | Query $\leq$ 1000 characters, <code>topK</code> $\leq$ 100, <code>rerankTopN</code> $\leq$ 20, <code>finalTopK</code> $\leq$ <code>rerankTopN</code> , JSON body $\leq$ 100 kB (413), malformed JSON (400) | <code>RetrievalValidationService.ts</code> , <code>app.ts</code>      |
| 18. Automated tests       | Regression safety            | Unit tests (validation, mapping, de-duplication, LLM output, fallbacks) and integration tests                                                                                                              | <code>npm test</code> , <code>npm run test:integration</code>         |
| 19. AI Search             | Product-ready results        | Person-level de-duplication, re-rank score and reason in results, pipeline counts, shortlist summaries endpoint (see Part E)                                                                               | <code>POST /v1/search</code> , <code>POST /v1/search/summaries</code> |

## 5. Part C — Web frontend

Project: `recruitbot-web/` . One commit per phase; each phase verified with a Playwright browser scenario before the next.

| Phase                   | Commit               | Implementation                                                                                                                                                                         |
|-------------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Setup                | <code>d509ae1</code> | Vite + React + TypeScript project, Tailwind theme, path aliases, ingestion feature folders, Vitest                                                                                     |
| 2. Upload UI            | <code>75aa436</code> | Drag-and-drop zone, file picker, Upload button, page layout                                                                                                                            |
| 3. API integration      | <code>fcea7af</code> | Axios client with request ids; <code>POST /v1/resume/inject</code> with form-data field <code>file</code> , 120 s timeout                                                              |
| 4. State store          | <code>d287676</code> | Zustand ingestion store: selected file, progress, result, typed error                                                                                                                  |
| 5. Progress stages      | <code>d8c98c4</code> | Upload $\rightarrow$ PDF processing $\rightarrow$ parsing $\rightarrow$ embedding $\rightarrow$ MongoDB $\rightarrow$ completed, driven by the real request state and returned timings |
| 6. Result screen        | <code>526d7fe</code> | Success and "already ingested" screens with file name and resume id                                                                                                                    |
| 7. File validation      | <code>2fc582d</code> | PDF only and 5 MB checks before upload (zod)                                                                                                                                           |
| 8. Error handling       | <code>b120bb0</code> | Errors mapped to the failed stage (extraction, parsing, embedding, storage, network) with Retry and Choose another file                                                                |
| 9. Ingestion checkpoint | <code>6f99477</code> | End-to-end check of the ingestion feature                                                                                                                                              |

| Phase                        | Commit  | Implementation                                                                                                       |
|------------------------------|---------|----------------------------------------------------------------------------------------------------------------------|
| 10. Retrieval infrastructure | 66c0b89 | UI primitives, stores, router ( / chat, /ingestion ), app shell                                                      |
| 11. Sidebar                  | 541e439 | Search modes, hybrid weight sliders and presets, results limit                                                       |
| 12. Chat interface           | dc9dea3 | Top bar, chat thread, suggestion chips, auto-growing input (Enter / Shift+Enter), mobile drawer                      |
| 13. Search and results       | eb2c71c | Adapter to /v1/search/vector , /bm25 , /hybrid ; client-side hybrid fusion (min-max scaling + weights); result cards |
| 14. Candidate profile        | c84d906 | Modal with contact, skills, experience, education from GET /v1/resumes/:id                                           |
| 15. Final checklist          | 3e73892 | Lazy-loaded pages and modal, accessibility fixes, deployment config; Lighthouse accessibility 100                    |

## 6. Part D — Rebrand and UX (commit e1ee54e)

| Change        | Details                                                                                                                                                                                                                                  |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Name and logo | RecruitBot → <b>TalentLens AI</b> ; supplied logo made transparent and used as wordmark (sidebar, upload page), mark (chat avatar, top bar) and favicon                                                                                  |
| Palette       | Light theme: Deep Navy #0B1F3A , Electric Blue #2563EB , Indigo #6366F1 , Emerald #10B981 , background #F8FAFC , cards #FFFFFF , text #0F172A / #64748B , borders #E2E8F0 ; darker text shades of the mode colours keep WCAG AA contrast |
| Guide         | /help page — quick start, search modes, how ingestion and search work, tips — linked next to the app name                                                                                                                                |
| Layout        | Clear moved next to Send (disabled on an empty chat; abandons a running search); larger blue search-mode badge                                                                                                                           |
| Fixes         | A search cleared while running no longer drops its reply into the empty thread; form fields given id / name                                                                                                                              |

## 7. Part E — AI Search enhancements (commit fc0e8a2)

Requirement: re-ranking, de-duplication of repeated results, summarization, end-to-end pipeline integration, clear result display, responsive design, and loading / success / validation / error messages.

### Backend

| Feature                     | Implementation                                                                                                                                                                                                                                                                 |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Person-level de-duplication | dedupeByPerson() : same normalised email, or phone (last 10 digits); name only when neither resume has contact details. Keeps the best-ranked resume, best scores, union of sources, and lists the others in duplicates . Found 6 people with 2 resume files each in the data. |
| Re-ranking details          | The re-ranker's relevanceScore (0–1) and reason are now returned with each result (null when re-ranking fell back)                                                                                                                                                             |
| Display fields              | bm25Score , vectorScore , email , phone , snippet , matchedSkills , duplicates per result; pipeline counts (retrieved, unique resumes, duplicates merged, re-ranked, returned) per response                                                                                    |

| Feature             | Implementation                                                                                                                                                                                       |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Shortlist summaries | POST /v1/search/summaries { query, resumeIds } — one Groq call returns an overall summary and a fit summary per candidate; data loaded server-side by id; unknown, repeated or too many ids rejected |
| Privacy             | Email and phone are used for de-duplication but never sent to the LLM                                                                                                                                |

## Frontend

| Feature                  | Implementation                                                                                                                                                                                                                           |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AI Search mode (default) | POST /v1/search (results limit = finalTopK, at least 10 candidates re-ranked); results shown first, summaries fetched afterwards                                                                                                         |
| Result display           | Pipeline header; AI summary card; cards with rank, relevance, "Keyword match" / "Semantic match", experience, contact, <b>Why this match</b> , fit summary, skills with query matches highlighted, "+ 1 more resume — Also on file: ..." |
| Loading                  | "Searching resumes, removing duplicates and re-ranking with AI..." with elapsed seconds; shimmer placeholders for summaries                                                                                                              |
| Validation               | Character counter from 800; over 1,000 characters an error message and Send disabled                                                                                                                                                     |
| Errors                   | Friendly message per backend error code, network error or timeout, with Retry search / Retry summary; no duplicate toasts                                                                                                                |
| Degraded results         | Amber notice when AI re-ranking, semantic or keyword search was unavailable                                                                                                                                                              |
| Responsive               | Wrapping chips and header, score below the name on narrow screens; no horizontal overflow at 375 px                                                                                                                                      |

## 8. API reference

| Method       | Endpoint                                                                       | Purpose                                     |
|--------------|--------------------------------------------------------------------------------|---------------------------------------------|
| GET          | /health                                                                        | Server health                               |
| POST         | /v1/resume/upload · /extract · /clean · /parse · /skills · /llm-parse · /embed | Ingestion steps one by one (form-data file) |
| POST         | /v1/resume/inject                                                              | Full ingestion of one PDF                   |
| POST / GET   | /v1/resume/batch-inject · /v1/resume/batch-status                              | Batch ingestion                             |
| POST         | /v1/resume/search                                                              | Phase 15 semantic search                    |
| GET / DELETE | /v1/resumes, /v1/resumes/:id                                                   | Resume management                           |
| GET          | /v1/search/readiness                                                           | Retrieval readiness                         |
| POST         | /v1/embeddings                                                                 | Query embedding                             |
| POST         | /v1/search/bm25 · /vector · /hybrid                                            | Retrieval strategies                        |
| POST         | /v1/search/rerank · /summarize                                                 | LLM re-ranking and single summaries         |
| POST         | /v1/search                                                                     | End-to-end pipeline (AI Search)             |

| Method | Endpoint             | Purpose                             |
|--------|----------------------|-------------------------------------|
| POST   | /v1/search/summaries | Shortlist + per-candidate summaries |

## 9. Verification summary

| Check                                                | Result                                |
|------------------------------------------------------|---------------------------------------|
| Backend unit tests ( npm test )                      | 48 / 48 pass                          |
| Frontend unit and component tests (Vitest)           | 56 / 56 pass                          |
| Type checks, lint (oxlint), production build         | Pass                                  |
| Browser scenario — AI Search (real backend and Groq) | 30 / 30 checks pass                   |
| Browser regression — frontend Phases 7–15            | All pass                              |
| Postman — retrieval Phase 19 (Newman)                | 7 requests, 21 / 21 assertions pass   |
| Lighthouse (production build, before AI Search)      | Accessibility 100 on / and /ingestion |

Known limits: Groq's free tier allows 8,000 tokens per minute per key; each AI Search uses two LLM calls, so a burst of searches can trigger the "re-ranking unavailable" notice or the summary Retry. Production deployment needs the API on the same origin (reverse proxy) or CORS on the backend.